Dominio 2 spiegato da zero: progettazione degli strumenti e integrazione MCP (18%)

Il dominio 1 riguardava il "cervello" dell'agente. Il dominio 2 riguarda le sue **mani**: gli strumenti che può usare, come vengono descritti, come si comportano quando qualcosa va storto, e come si collegano a servizi esterni.

> **Prima di iniziare.** Questo è il dominio con più concetti tecnici "di contorno". Quando incontri → **Pre-corso** trovi il link alla spiegazione completa. Per questo dominio conviene aver letto almeno:
>
> - Funzioni, parametri e valori di ritorno (uno strumento è la descrizione di una funzione);
> - JSON e JSON Schema (come si descrivono gli input di uno strumento);
> - Richiesta HTTP, con la tabella dei codici di stato (`200`, `400`, `429`, `503`...);
> - Client e server e Variabili d'ambiente (per la parte su MCP).
>
> Per i termini specifici di Claude tieni aperto il Vocabolario.

---

1. Lo strumento è un annuncio di lavoro

Quando Claude deve decidere quale strumento usare, non può vedere il codice dietro. Legge solo il **nome** e la **descrizione** di ciascuno, e poi sceglie.

Pensa a una selezione di personale: se l'annuncio dice "Cerca elementi", chiunque risponde con una scelta a caso. Se dice "Cerca nella knowledge base interna articoli di supporto e FAQ. Usalo quando il cliente pone una domanda che potrebbe avere una risposta documentata. Restituisce titoli, estratti e ID", la scelta è chiara.

Com'è fatto uno strumento

→ Pre-corso: Funzioni, parametri e valori di ritorno, con la tabella che confronta funzione e strumento.

Uno strumento, per Claude, è una scheda con tre parti:

```
{
  "name": "search_knowledge_base",
  "description": "Cerca nella knowledge base interna articoli di supporto e FAQ.
Usalo quando il cliente pone una domanda che potrebbe avere una risposta
documentata (procedure, policy, istruzioni). Non usarlo per dati personali del
cliente: per quelli usa get_customer. Restituisce fino a 5 risultati con titolo,
estratto e ID. Se non trova nulla restituisce una lista vuota.",
  "input_schema": {
    "type": "object",
    "properties": {
      "query": { "type": "string", "description": "Domanda o parole chiave del
cliente" }
    },
    "required": ["query"]
  }
}
```

- `name`: il nome, breve e senza ambiguità;
- `description`: l'annuncio di lavoro, scritto in linguaggio naturale;
- `input_schema`: il modulo da compilare, scritto in JSON Schema (quali campi, di che tipo, quali obbligatori).

Non serve saper scrivere questa scheda a memoria. Serve capire che **tutto ciò che Claude sa dello strumento è qui dentro**.

Cosa deve dire una buona descrizione

Una buona descrizione dice:

- **cosa fa**;
- **quando usarlo**;
- **quale input richiede**;
- **cosa restituisce**;
- **cosa succede nei casi limite**;
- **quando usare invece uno strumento simile**.

Il rischio concreto è questo: se due strumenti hanno descrizioni quasi identiche, come `analyze_content` e `analyze_document`, Claude sbaglierà spesso. La correzione è rinominarli in modo da distinguerli, oppure unirli in un unico strumento.

Nelle domande d'esame, quando Claude "sceglie lo strumento sbagliato", la prima correzione da considerare è quasi sempre **migliorare nomi e descrizioni**, non aggiungere istruzioni nel prompt generale.

Il limite: 4 o 5 strumenti per agente

Un agente con 18 strumenti sceglie male. Più opzioni significano più confusione. La regola pratica è 4–5 strumenti per agente: se ne servono di più, dividili tra subagenti specializzati.

Pensa a un menu di ristorante con 200 piatti: si sceglie peggio e più lentamente che con un menu di 10. In più, ogni definizione di strumento occupa spazio nel contesto a ogni richiesta (vedi il dominio 5).

→ Pre-corso: Privilegio minimo. Dare a ogni agente solo gli strumenti che gli servono è anche una scelta di sicurezza: un agente che deve solo leggere non dovrebbe avere uno strumento per cancellare.

Lo schema e lo "strict mode"

Ogni strumento dichiara quali input accetta, con uno **schema** (una specie di modulo con campi e tipi). Con `strict: true` Claude è obbligato a rispettare il modulo: niente campi inventati, tipi corretti.

→ Pre-corso: JSON Schema, per capire `type`, `properties`, `required` ed `enum`.

Per attivarlo, lo schema deve rispettare due regole:

- `additionalProperties: false`, cioè non sono ammessi campi extra;
- tutte le proprietà devono comparire in `required`, cioè tutte obbligatorie.

E se un campo è davvero facoltativo? Lo si rende obbligatorio ma "annullabile" (`nullable`): il campo c'è sempre, ma può valere `null`. Il dominio 4 riprende questo punto.

---

2. Quando qualcosa va storto: gli errori

Un errore utile è come una segnalazione precisa: non dice solo "qualcosa non va", ma dice **che cosa** non va, e se si può riprovare.

→ Pre-corso: Richiesta HTTP (i codici di stato) e Rate limit, timeout, retry e backoff.

Le categorie da conoscere:

Tipo di errore	Esempio	Si può riprovare?	Cosa fare
Temporaneo	Timeout, troppe richieste in poco tempo	Sì	Riprova aspettando un po' di più ogni volta
Convalida	Manca un campo, formato sbagliato	No, senza correggere l'input	Correggi e riprova
Business	Il rimborso supera il limite della policy	No	Spiega la regola o passa a una persona
Autorizzazione	Il token di accesso è scaduto	No	Passa a una persona

Come riconoscerli dai codici HTTP, quando lo strumento chiama un servizio web:

Codice	Di solito è un errore...
`429`, `503`, timeout	temporaneo
`400`	di convalida
`401`, `403`	di autorizzazione
`409` o una risposta con un messaggio di regola violata	di business

Come si scrive un errore utile

Confronta:

```
{ "error": "Operation failed" }
```

```
{
  "error_type": "business",
  "message": "Il rimborso di 750 € supera il limite automatico di 500 €.",
  "retryable": false,
  "suggestion": "Proponi al cliente il passaggio a un operatore con
escalate_to_human."
}
```

Il primo non permette a Claude di fare nulla di sensato. Il secondo gli dice cosa è successo, che non ha senso riprovare e qual è la strada alternativa. Il risultato va restituito con `is_error: true` (vedi il dominio 1).

→ Pre-corso: Idempotenza. Anche un errore "temporaneo" va riprovato con cautela se l'azione non è idempotente: ripetere un pagamento andato in timeout può addebitarlo due volte.

Il concetto più importante: "vuoto" non è "errore"

Qui c'è la distinzione più verificata all'esame.

- Una ricerca che **funziona** e non trova nulla restituisce, per esempio, lo stato `200` (tutto ok) con una lista vuota. Significa: "ho cercato, non c'è niente". Claude può dire: "Non ho trovato articoli".
- Una ricerca che **non parte** perché il servizio è giù (timeout, errore `503`) significa: "non ho potuto cercare". Se Claude risponde "non ho trovato nulla", dice una bugia involontaria.

Un'analogia: chiedi in biblioteca se c'è un libro. La bibliotecaria controlla lo scaffale e dice "non c'è". Ma se il computer del catalogo era spento, la risposta corretta è "non riesco a controllare adesso", non "non c'è".

Perché è così grave confonderli? Perché il cliente prende decisioni sulla risposta. Se gli dici "non risulta nessun ordine a tuo nome" quando in realtà il gestionale era irraggiungibile, il cliente può rifare l'ordine (e pagarlo due volte) o pensare a una truffa.

Gli antipattern da riconoscere

- **Errore generico**: `"Operation failed"`. Non dice cosa è successo, quindi Claude non può recuperare.
- **Soppressione silenziosa**: restituire una lista vuota quando in realtà il servizio è inattivo. Nasconde il problema.
- **Interruzione totale**: se un passaggio fallisce, interrompere tutto il workflow anche quando i risultati parziali sarebbero utili.

→ Pre-corso: Fallimento parziale e degradazione controllata.

---

3. Come Claude decide se usare uno strumento: `tool_choice`

Questo è uno dei punti più tecnici e più chiesti. Il parametro `tool_choice` (letteralmente "scelta dello strumento") è un'impostazione che lo sviluppatore aggiunge alla richiesta. Risponde a due domande separate:

1. **Claude è obbligato a usare uno strumento in questo turno?** Sì o no.
2. **Se lo usa, chi sceglie quale?** Claude, oppure lo sviluppatore?

Combinando le due domande ottieni quattro valori. Immagina un cameriere che deve prendere un ordine:

Valore	Traduzione	Può rispondere solo a parole?	Chiamata garantita?	Chi sceglie lo strumento?
`auto` (predefinito)	"Decidi tu se serve"	Sì	No	Claude
`any`	"Devi usare uno strumento, uno a tua scelta"	No	Sì	Claude
`tool` con nome	"Devi usare proprio questo"	No	Sì	Lo sviluppatore
`none`	"Non usare strumenti"	Sì, obbligatoriamente	No, vietate	Nessuno

Per scegliere, chiediti: "Claude deve usare uno strumento in questo turno?"

- "Forse, decide lui" → `auto`.
- "Sì, ma scelga lui quale" → `any`.
- "Sì, e deve essere questo preciso" → `tool` con il nome.
- "No, solo testo" → `none`.

Esempi pratici:

Situazione	Valore
Assistente clienti: a volte basta rispondere, a volte serve cercare l'ordine	`auto`
Classificatore di documenti con tre strumenti di estrazione diversi, uno per tipo di documento: deve sempre usarne uno	`any`
Pipeline che deve sempre restituire i metadati con `extract_metadata`	`tool` con nome
Turno finale in cui Claude deve solo scrivere il riepilogo per il cliente	`none`

Le confusioni tipiche

1. **`auto` non garantisce niente.** Se una domanda dice "garantisce una chiamata", `auto` non è la risposta.
2. **`any` e `tool` obbligano entrambi, ma differiscono nella scelta.** "Sempre `extract_metadata`" è `tool`. "Uno dei tre strumenti di estrazione, a scelta" è `any`.
3. **`none` non equivale a togliere gli strumenti.** Vieta le chiamate, ma le definizioni restano nel contesto. Il vantaggio: modificare la lista degli strumenti invalida la cache (la memoria veloce di cui parliamo nel

dominio 4), mentre `none` no. Quindi, se un turno deve produrre solo testo ma gli strumenti servono dopo, usa `none`.
4. **`disable_parallel_tool_use` non è un valore di `tool_choice`.** È un'opzione che si aggiunge a qualsiasi valore e limita Claude a una sola chiamata per risposta. Non decide se la chiamata avviene.

→ Pre-corso: Cache, per capire perché "invalidare la cache" costa tempo e denaro.

Un avviso sui modelli recenti

Nel materiale si dice che alcuni modelli più recenti (Fable 5.1, Mythos 5.1 e Opus 5.5) **rifiutano** `any` e la forma forzata `tool` con errore `400`. Sonnet 5 le supporta ancora, e le domande d'esame ragionano di solito su quel modello. Per ottenere un output strutturato sui modelli recenti, la strada indicata è `auto` con un'istruzione esplicita e `strict: true`, oppure gli output strutturati (di cui si parla nel dominio 4).

---

4. MCP: il collegamento con il mondo esterno

MCP (Model Context Protocol) è uno standard che permette a Claude di collegarsi a servizi esterni: database, sistemi aziendali, strumenti di terze parti. Immagina una presa universale: se il servizio ha la spina MCP, Claude può usarlo senza che tu scriva un adattatore su misura per ciascuno.

Perché serve uno standard

Senza MCP, ogni applicazione dovrebbe scrivere il proprio collegamento verso ogni servizio: il CRM, GitHub, il database, il calendario. Con dieci applicazioni e dieci servizi servirebbero cento collegamenti diversi. Con uno standard, ogni servizio scrive **un** server MCP e ogni applicazione sa parlare MCP: i collegamenti diventano riutilizzabili.

È la stessa idea della porta USB: prima ogni dispositivo aveva il suo cavo, oggi una porta va bene per tutti.

→ Pre-corso: API e REST. Molti server MCP sono un "involucro" attorno a un'API già esistente: traducono le sue operazioni in strumenti che Claude può usare.

Cosa offre un server MCP

Un server MCP può esporre tre cose:

- **strumenti** (*tools*): azioni che Claude può chiedere di eseguire, come "crea un ticket";
- **risorse** (*resources*): dati che si possono leggere e mettere nel contesto, come un documento o lo schema di un database;
- **prompt**: modelli di istruzioni già pronti, che l'utente può richiamare.

I tre ruoli

→ Pre-corso: Client e server

- **Host**: il programma che usi, per esempio Claude Code. Gestisce le connessioni.
- **Client**: un collegamento dedicato dentro l'host, uno per ogni server. Ogni client parla con un solo server.
- **Server**: il servizio che espone i propri strumenti, risorse e prompt.

Una metafora: l'host è un centralino, ogni client è una linea telefonica dedicata, ogni server è un ufficio all'altro capo della linea.

I modi di collegarsi (trasporti)

→ Pre-corso: Processo, subprocess e stdio e Chiave API e autenticazione (per OAuth).

- `stdio`: il server gira sullo stesso computer, come un programma avviato localmente. È il più comune.
- `http` (streamable-http): il server è remoto, su internet. È l'unico tipo remoto che supporta l'autenticazione OAuth (il meccanismo "accedi con il tuo account").
- `sse`: sistema più vecchio, considerato legacy.
- `ws`: connessione persistente, con autenticazione solo tramite intestazione.

In pratica: **locale** (sul tuo computer, avviato da Claude Code) → `stdio`; **remoto** (un servizio online) → `http`.

Errore di configurazione comune: una voce con un indirizzo `url` ma senza il campo `type`. Viene ignorata.

Dove si registra un server: gli ambiti

Stesso server, quattro posti diversi a seconda di chi deve usarlo:

- **Locale** (predefinito): solo per il progetto corrente e solo per te, salvato in `~/.claude.json`. Non finisce nel repository condiviso.
- **Progetto**: nel file `.mcp.json` nella radice del progetto. È condiviso con il team tramite Git.
- **Utente**: per tutti i tuoi progetti, in `~/.claude.json`.
- **Gestito**: impostato dall'amministratore, non rimovibile dagli utenti.

→ Pre-corso: Repository e Git, per capire cosa significa "condiviso con il team tramite Git".

Come scegliere: se **tutto il team** deve avere lo stesso server (per esempio il database di sviluppo del progetto) → **progetto**. Se è uno strumento che usi **solo tu** in tutti i tuoi progetti → **utente**. Se lo stai provando solo in questo progetto → **locale**.

Le credenziali: mai nel file

→ Pre-corso: Chiave API e autenticazione e Variabili d'ambiente.

Le password e le chiavi d'accesso non vanno scritte in `.mcp.json`, perché quel file viene condiviso. Si scrive un segnaposto, `${DATABASE_URL}`, e il valore vero sta nell'ambiente del computer, fuori dal progetto. Il file da condividere resta pulito.

Esempio di `.mcp.json`:

```
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["@modelcontextprotocol/server-postgres"],
      "env": { "DATABASE_URL": "${DATABASE_URL}" }
    }
```

```
    }
  }
```

Come si legge, riga per riga:

- `"postgres"` è il nome che diamo al server;
- `"command": "npx"` e `"args"` dicono a Claude Code quale programma avviare (qui un server MCP per il database PostgreSQL, scaricato ed eseguito al volo con `npx`);
- `"env"` passa al server una variabile d'ambiente: `${DATABASE_URL}` viene sostituito con il valore presente sul computer di ciascuno.

→ Pre-corso: Librerie, SDK e pacchetti (cos'è `npx`) e Database.

Approvazione automatica: attenzione

Nelle sessioni interattive, Claude Code chiede il permesso prima di usare un server definito in `.mcp.json`. Nelle sessioni **non interattive** (quelle lanciate da script, da pipeline o dal cloud) non c'è nessuno che possa rispondere, quindi il server viene caricato senza chiedere. Se non lo vuoi, va escluso esplicitamente.

→ Pre-corso: CI/CD e pipeline, per capire cosa sono le sessioni "senza nessuno davanti".

---

5. Gli strumenti integrati e quelli "lato server"

Claude Code ha strumenti propri, che conviene conoscere per nome:

- `Grep` cerca un testo nei file; `Glob` cerca file per nome.
- `Read` legge un file.
- `Edit` modifica un file con una sostituzione mirata: il testo da cambiare deve essere **unico** nel file, altrimenti non sa dove intervenire.
- `Write` crea un file nuovo.
- `Bash` esegue comandi.
- `Agent` lancia un subagente.

→ Pre-corso: File system, percorsi e glob, Regex (`Grep` accetta espressioni regolari) e Terminale, shell e CLI (cosa fa `Bash`).

Quale usare? Una tabella di scelta rapida:

Devo…	Strumento
trovare tutti i file di test	`Glob` (per esempio `**/*.test.ts`)
trovare dove viene usata la funzione `calcolaIva`	`Grep`
capire cosa fa un file	`Read`
cambiare una riga in un file esistente	`Edit`
creare un file nuovo	`Write`
eseguire i test o un altro comando	`Bash`

Devo…	Strumento
esplorare a fondo senza riempire la conversazione	`Agent` (subagente)

Per esplorare una codebase (l'insieme dei file di un progetto software) il metodo corretto è: prima `Grep` per trovare i punti d'ingresso, poi leggi quei file, poi segui le importazioni. Leggere tutto subito è un antipattern: riempie il contesto di file irrilevanti (vedi il dominio 5).

Strumenti lato server

Strumenti lato server (per esempio la ricerca web): li esegue Anthropic, non il tuo programma. Il risultato arriva nella stessa risposta. Attenzione: un errore **non** fa scattare un'eccezione, arriva come risposta normale con stato `200`. Quindi il codice deve controllare il contenuto.

In altre parole: "la chiamata HTTP è andata bene" non significa "la ricerca è andata bene". È lo stesso tema del "vuoto non è errore": bisogna guardare dentro la risposta.

---

6. Il dettaglio che fa perdere punti: i `tool_result` in parallelo

Quando Claude chiede più strumenti nello stesso messaggio, tu devi:

- eseguirli tutti;
- restituire **tutti** i risultati **in un unico messaggio** utente.

Se li mandi in messaggi separati, Claude impara a non fare più chiamate parallele, e il sistema rallenta.

E se uno strumento fallisce, **non omettere mai** il suo risultato: invia un `tool_result` con `is_error: true`. Un'omissione lascia Claude senza risposta e può bloccare il loop.

Esempio: Claude chiede insieme `get_customer`, `lookup_order` e `check_shipping`. Il terzo va in timeout. Il messaggio di ritorno contiene **tre** risultati, ciascuno con il suo `tool_use_id`: due con i dati e uno con `is_error: true` e la spiegazione del timeout. Claude può così rispondere con ciò che ha e dire cosa manca.

---

7. Le cinque cose da ricordare

1. La **descrizione** decide la scelta: dice quando usare e quando no.
2. **Errore** e **risultato vuoto** sono cose diverse.
3. Al massimo **4–5 strumenti** per agente.
4. Le **credenziali** stanno nell'ambiente, mai nel file di configurazione.
5. Tutti i **risultati paralleli** in un solo messaggio.

Per capire meglio: come ragionare sulle domande

Quando una domanda parla di "garantire una chiamata", pensa alla tabella di `tool_choice`. Quando parla di un risultato vuoto e di un servizio che non risponde, ricordati la biblioteca: "non c'è" è diverso da "non riesco a controllare". Quando parla di credenziali, la risposta è sempre quella dell'ambiente, mai del file condiviso.

Tre ragionamenti tipici

"Claude usa spesso `analyze_document` quando dovrebbe usare `analyze_content`." Il problema è nelle descrizioni troppo simili. La correzione è distinguerle (o unire gli strumenti), non aggiungere un esempio nel prompt di sistema.

"Il servizio di ricerca era giù, ma l'agente ha detto al cliente che non esistevano articoli." Lo strumento ha restituito una lista vuota invece di un errore. La correzione è restituire un errore temporaneo esplicito, con `is_error: true`.

"Il team deve condividere un server MCP per il database, ma ognuno ha la propria password." Server nel file `.mcp.json` di progetto, password come variabile d'ambiente `${…}` sul computer di ciascuno.